

Esercizio_Server di Rifornimento Razzi Multi-Thread con Mutex

Immagina di programmare il software di controllo di una stazione spaziale che deve rifornire di propellente i razzi cargo prima che partano per Marte. La stazione ha una cisterna principale con una quantità limitata di carburante totale (es. **5000 litri**) e **tutti i razzi (client) che si connettono attingono contemporaneamente dalla stessa cisterna**.

Logica del Server Concorrente

- **Variabili Globali:**
 - Un intero `carburanteDisponibile = 5000;` (la risorsa condivisa).
 - Un mutex `pthread_mutex_t mutexCarburante = PTHREAD_MUTEX_INITIALIZER;` (la protezione per la cisterna).
- **Il Main Thread:**
 - Accetta le connessioni dei razzi in un ciclo `for(;;)` e lancia un thread dedicato per ogni razzo che si aggancia alla stazione, passandogli il socket.
- **Il Thread di Gestione Rifornimento (Logica Worker):**
 1. Il razzo appena connesso invia un intero che rappresenta i **litri di carburante richiesti** per il suo viaggio.
 2. Il thread riceve la richiesta e deve controllare/modificare la variabile globale `carburanteDisponibile`.
 3. **La Sezione Critica:** Prima di toccare la cisterna globale, il thread deve bloccare il mutex, verificare la disponibilità, aggiornare il contatore e poi sbloccarlo immediatamente:
 4. Il server risponde al razzo inviando l'esito della manovra tramite una stringa: `"RIFORNIMENTO OK"` oppure `"PROPULLENTE INSUFFICIENTE"`.
 5. Il thread chiude il socket (il razzo si sgancia) e termina.

Logica del Client (Il Razzo Cargo)

Il client si connette, fa rifornimento e si scollega subito dopo:

1. Si connette alla stazione spaziale (Server).
2. Chiede al comandante (tramite terminale): `Quanti litri di propellente servono per il lancio? ->` .
3. Invia il numero intero al server.
4. Riceve la stringa di risposta dalla stazione e la stampa a schermo.
5. Si disconnette e termina l'esecuzione.